

Data Structures and Algorithms (DSA)

Study Material for AI Quiz Generation

Arrays

An array stores elements in contiguous memory locations. Elements can be accessed directly using an index in $O(1)$ time. Searching an unsorted array takes $O(n)$. Inserting or deleting in the middle can take $O(n)$ because elements may need to be shifted.

Linked Lists

A linked list consists of nodes containing data and links. A singly linked list points to the next node; a doubly linked list has previous and next links. Accessing an arbitrary node takes $O(n)$. Insertion or deletion can be $O(1)$ when the required pointer is already available.

Stack

A stack follows LIFO (Last In, First Out). Main operations are push, pop, and peek/top. With a suitable implementation, these operations take $O(1)$ time.

Queue

A queue follows FIFO (First In, First Out). Enqueue adds an element and dequeue removes an element. With a suitable implementation, both can take $O(1)$ time.

Hash Tables

A hash table stores key-value pairs using a hash function. Average insertion, deletion, and lookup are $O(1)$. Collisions can be handled using chaining or open addressing.

Binary Search

Binary search works on sorted data. It compares the target with the middle element and discards half of the search space after each comparison. Time complexity is $O(\log n)$.

Sorting

Bubble sort, selection sort, and insertion sort have $O(n^2)$ average/worst-case time in their standard forms. Merge sort has $O(n \log n)$ time. Quick sort has average $O(n \log n)$ and worst-case $O(n^2)$ time.

Trees

A binary tree has at most two children per node. A Binary Search Tree has smaller values in the left subtree and larger values in the right subtree. Inorder traversal of a BST visits values in sorted order. Common traversals are preorder, inorder, and postorder.

Heaps

A heap is a complete binary tree satisfying a heap property. In a min-heap the minimum is at the root; in a max-heap the maximum is at the root. Root access is $O(1)$, while insertion and root deletion are $O(\log n)$.

Graphs

A graph contains vertices and edges. It can be directed or undirected and weighted or unweighted. Graphs can be represented with adjacency lists or matrices. BFS uses a queue; DFS uses recursion or a stack. With adjacency lists, BFS and DFS take $O(V + E)$.

Recursion

Recursion is a technique where a function calls itself on smaller instances. A recursive algorithm needs a base case and a recursive case. Recursive calls use call-stack memory.

Dynamic Programming

Dynamic programming solves problems with overlapping subproblems and optimal substructure by storing results. Memoization is top-down; tabulation is bottom-up.

Greedy Algorithms

A greedy algorithm repeatedly makes a locally best choice. Greedy methods work for problems where the greedy-choice property and optimal substructure apply.

Big-O

Big-O describes the asymptotic growth of an algorithm. Common complexities are $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, and $O(n^2)$.

Quick Revision

- Array index access — $O(1)$
- Linear search — $O(n)$
- Binary search — $O(\log n)$
- Stack — LIFO
- Queue — FIFO
- BFS — Queue
- DFS — Stack/recursion
- Merge sort — $O(n \log n)$
- Quick sort — average $O(n \log n)$, worst $O(n^2)$
- Hash table lookup — average $O(1)$
- BST inorder traversal — sorted order
- Heap root access — $O(1)$